TECHNICAL REPORT

Email Response Agent: System Design and Security Analysis

Email Response Agent project

November 2026 · Repository `dhiasalah/email-response-agent` · Code state `main@adcaf73`

Abstract. We describe the design and security architecture of the Email Response Agent, a multi-agent system that drafts replies to a university professor's email while asking the professor for each missing fact only once. Each fact a reply depends on is represented as a typed *input slot*. Emails waiting on the same slot are grouped, and a single answer from the owner completes all of them.

Six single-purpose agents are coordinated by a deterministic orchestrator written in plain code. No language model holds a capability to send, forward or store mail. The security design layers deterministic controls ahead of every model call: pre-filtering, HTML and Unicode sanitisation, sender authentication based on the receiving server's DMARC and DKIM verdicts, and a rule-based prompt-injection detector. Deterministic reply rules follow, which no model verdict can overrule, and the database itself enforces the email lifecycle and outbox invariants.

We report 232 automated tests and an end-to-end corpus of 17 attacks mapped to the OWASP Top 10 for LLM Applications (2025), all passing on the current code. Last, we discuss the residual risks identified during an independent code review.

Keywords: multi-agent systems, large language models, prompt injection, email authentication, DMARC, DKIM, privacy by design.

1 Introduction

A professor receives many emails that differ in wording but wait on the same piece of information: the publication date of a grade, the room of an exam, a project deadline. Answering them one by one wastes time. Delegating them to a general-purpose language model exposes students to invented facts and the owner's mailbox to prompt injection [6]. The Email Response Agent answers recurring questions once, and never states a fact that was not provided by the owner or a trusted source.

This report has two aims. Sections 2 and 3 document the architecture as built. Sections 4 to 6 analyse its security and privacy properties and the evidence for them. Section 7 lists known limitations. Every figure and table was checked against the code at commit `adcaf73`; the source of each is named in its caption.

2 System overview

The unit of work is the input slot, a triple of *type*, *entity* and *qualifier* drawn from a fixed taxonomy of nine slot types: grades publication date, exam date, exam room, deadline, procedure, document availability, meeting availability, individual decision and individual result. Triage splits each email into questions and extracts their slots. The resolver tries to fill each slot from trusted sources in a fixed order: scoped knowledge memory, the knowledge base, the

calendar, and finally the owner's replies from the last seven days. Slots that remain unresolved are clustered into *input cards*. The owner answers each card once, and every linked email receives its own reply in the sender's language (Figure 1).

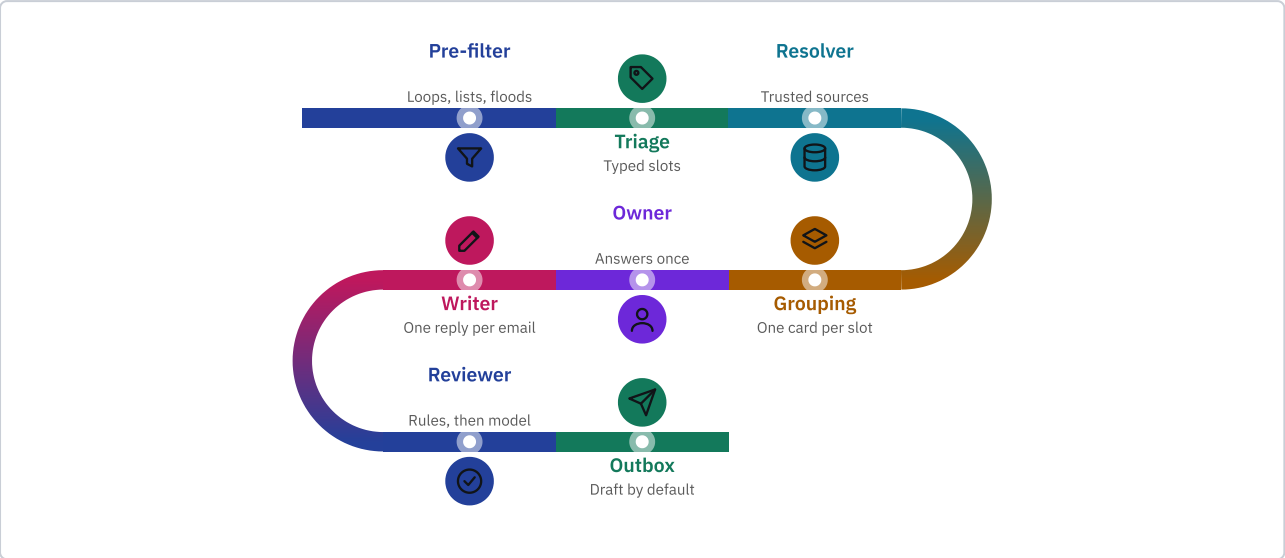


Figure 1. Processing pipeline. Every arrow is a lifecycle transition checked by the database (Figure 2). Source: `orchestrator/*Step*.java`, `agents/*Agent.java`.

Answers become *facts* with a scope (program, track, year), an expiry date and a shareable flag. A later sender in the same scope receives the answer without involving the owner. A sender in another scope, or with an unknown scope, triggers a new card instead.

3 Architecture

3.1 Deterministic orchestration

The orchestrator is ordinary Java code rather than an LLM, so routing is predictable, testable and cheap. It moves each email through a fixed set of fourteen statuses (Figure 2). Each step is keyed by email identifier, status and version, so a restart cannot repeat a side effect. A PostgreSQL trigger, `email_status_transition()`, rejects any transition outside the permitted set. A second set of triggers refuses an outbox row unless a passed review exists for the exact draft being sent.

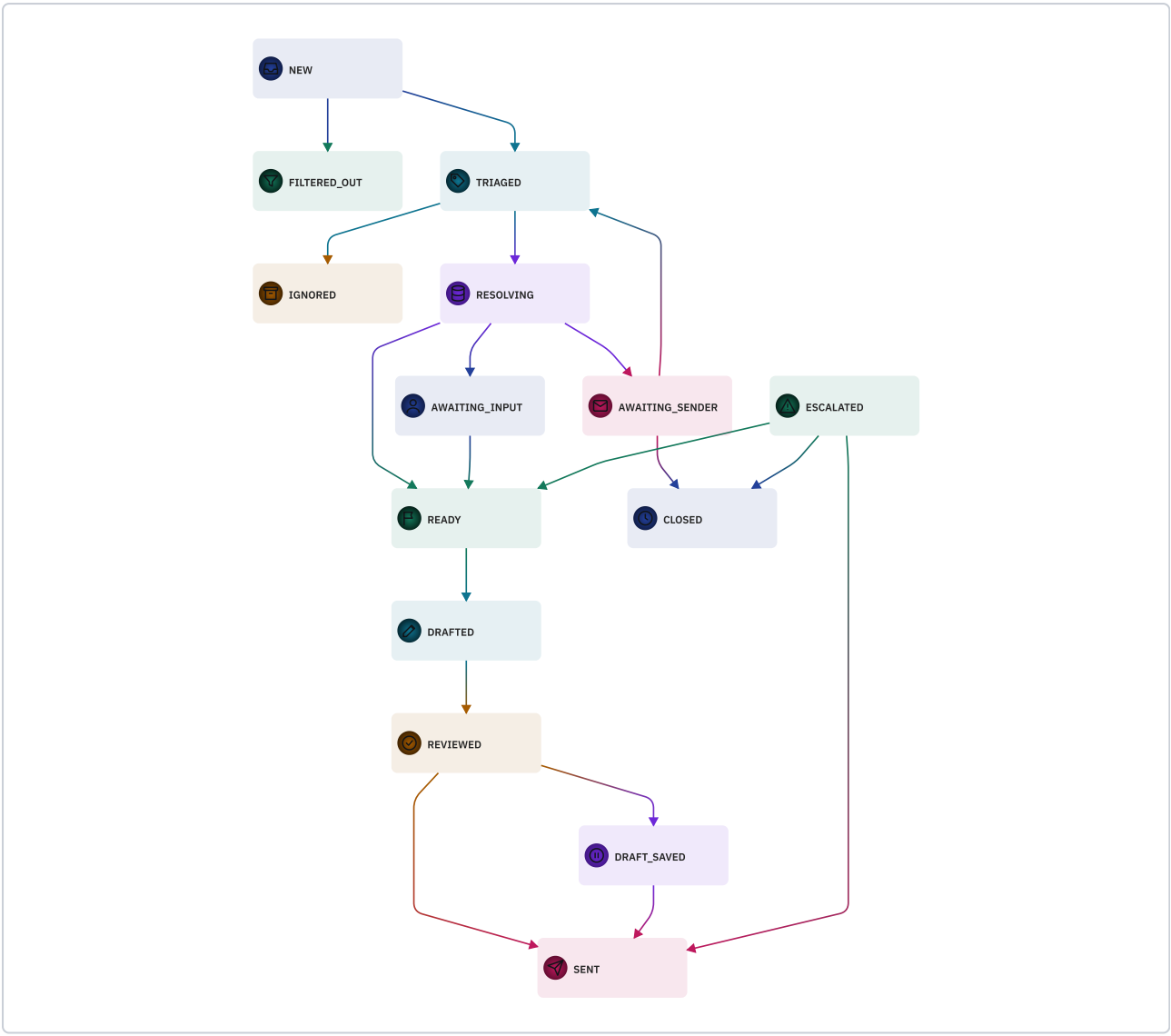


Figure 2. Email lifecycle. The 18 transitions shown, plus the three leaving `ESCALATED`, are exactly those the database trigger permits. Each of the nine working statuses may also move to `ESCALATED`; these nine edges are omitted for legibility. Source: `db/migration/V1701__owner_replies.sql`.

3.2 Agents

Six agents, each with one responsibility, communicate only through the shared state (Figure 3, Table 1). Every model output is parsed against a strict schema. An invalid output is retried once with the validation error, and a second failure escalates the email to the owner. Models run locally through Ollama at temperature 0. A fast tier serves classification and clustering, and a strong tier serves writing and review.

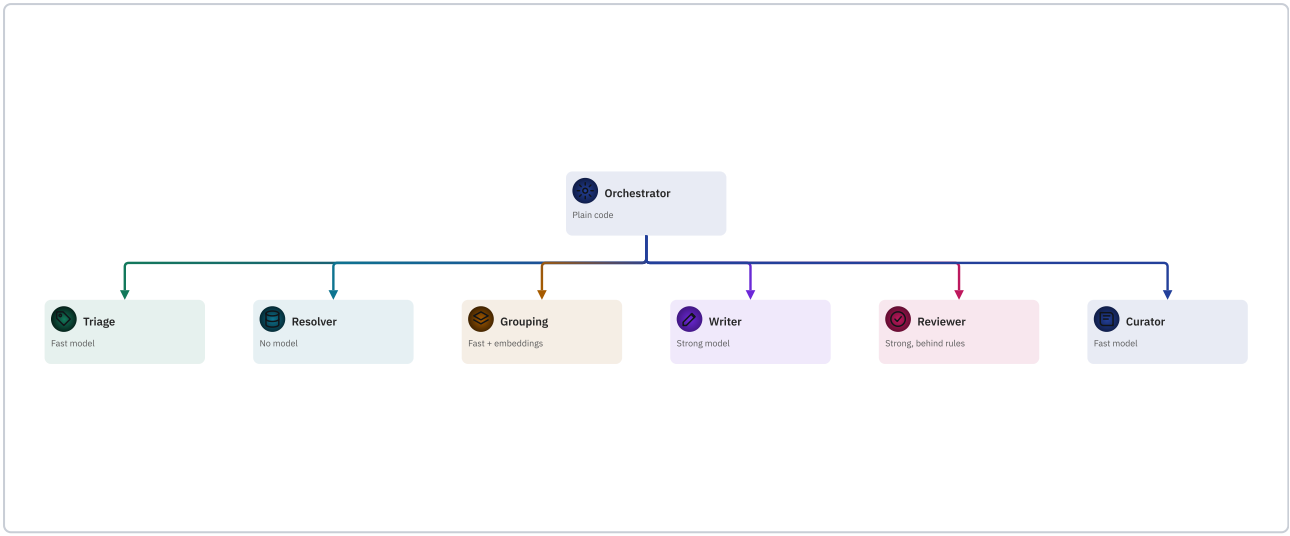


Figure 3. Agents and their model tiers under the orchestrator. Agents never call each other. Source: `agents/`, `llm/ModelTier.java`.

Table 1. Agent specifications as implemented.

Agent	Input	Output	Model tier
Triage	Sanitised email, sender scope from the roster	Intent, sensitivity, language, typed slots with owner	Fast
Resolver	Slots, sender scope	Each slot filled, proposed or unresolved, with source	None (code)
Grouping	Unresolved slots, open cards	Input cards; embedding candidates verified by the model	Fast + embeddings
Writer	Email, filled slots, owner note	Reply body and the list of facts used	Strong
Reviewer	Draft, facts used, policy	Pass or fail, outlier reasons	Strong, behind code rules
Curator	Card answers, sent replies	Scoped, expiring facts; recent-answer log	Fast

3.3 Owner interaction

The owner works from a local review panel with three queues: input cards ranked by priority, drafts grouped per card, and escalations. Card priority is a weighted sum of urgency (0.4), volume (0.3), age (0.2) and sender role (0.1). Drafts of a card are validated once through a representative reply, chosen by majority language and median length. Outliers, such as a writer retry, an unanchored yes/no fact or suspicious input, are shown individually and are never approved in bulk.

4 Security design

4.1 Threat model

Every incoming email is attacker-controlled data, headers included. The adversary can write any body, subject, display name or `From` address. The adversary may know the system's prompts, and may control what the language model outputs. The goals considered are:

- making the system send false or unauthorised information, including promises;
- exfiltrating data or redirecting students;

- obtaining another cohort's or another student's answers;
- exhausting the system or the owner.

Table 2 maps these threats to the OWASP Top 10 for LLM Applications [6] and to the controls that address them.

Table 2. Threats, OWASP LLM (2025) categories and implemented controls.

Threat	OWASP	Controls
Instructions hidden in an email	LLM01	Sanitiser; rule-based detector scored before any model call; the model may only raise the score
Invented facts, dates or promises	LLM09	Reply rules: every number, link and commitment word must appear in a cited fact
Data leaks, sensitive topics	LLM02	Personal slot types never reused or sent automatically; blocked-topic and grade rules
Markup or header injection in output	LLM05	Plain-text replies; one-line subject; recipient and thread taken from the email, never from the model
Agent acting beyond its role	LLM06	No send, forward or write tools for models; database-checked outbox; scope from a roster, never from the email
Loops, floods, cost exhaustion	LLM10	Auto-reply filter [5]; 10 emails per sender per hour; 3 automatic replies per sender and 50 in total per day; bounded batches
Forged sender identity	LLM06	Sender authentication from DMARC/DKIM verdicts (Section 4.3)

4.2 Defence in depth

Eight layers separate an incoming email from a sent reply (Figure 4). The first four are deterministic and run before any model sees the email. The review rules run on the final reply text, and no model verdict can overrule them. The last layer lives in the database and holds even if the application code were wrong.

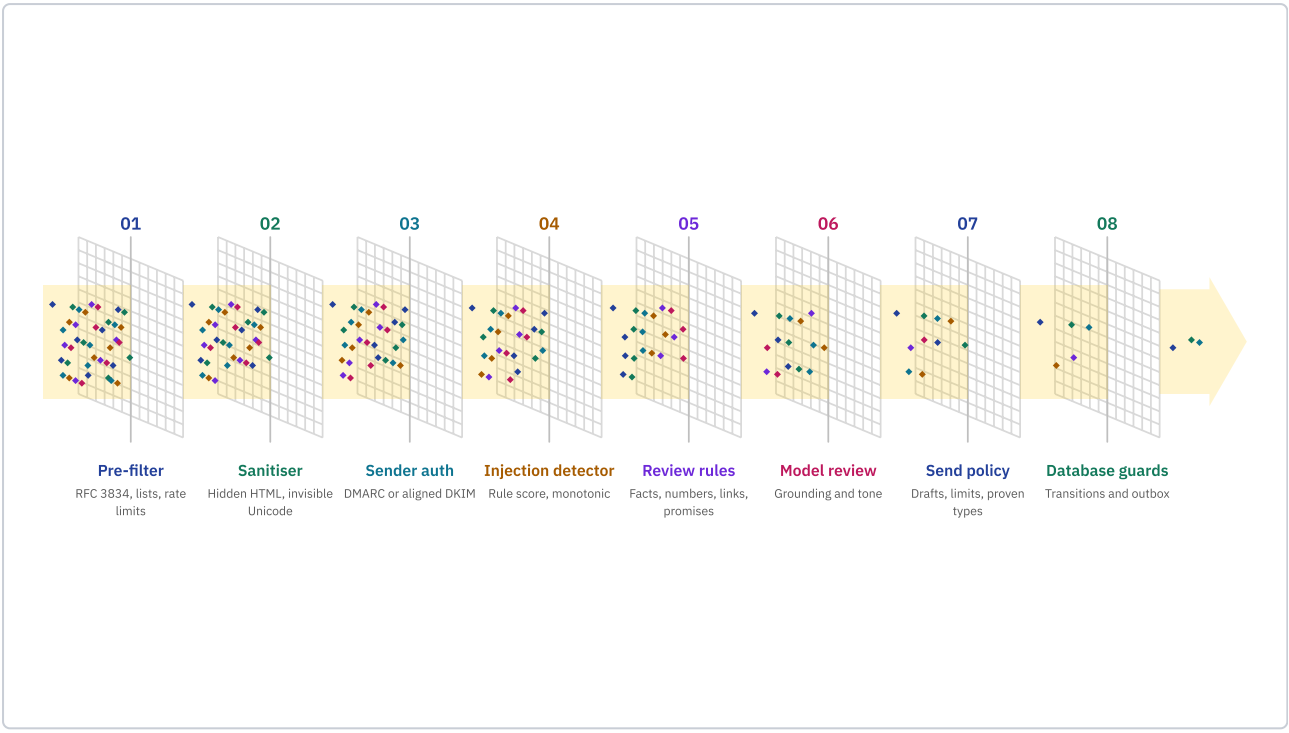


Figure 4. Defence layers, in the order an email meets them. Source: `security/`, `agents/ReviewRules.java`, `orchestrator/SendDispatcher.java`, migrations V1101 and V1701.

The injection detector combines weighted text rules with the sanitiser's flags. Examples are overridden instructions (1.0), spoofed roles (1.0), addressing the AI (0.6), an exfiltration address (0.6), hidden HTML (1.0) and an unauthenticated sender (0.3). It escalates at a score of 1.0, and the model's opinion is monotonic: it can add 0.6 but never subtract. The deterministic reply rules reject:

- a missing fact, an unsupported number, link or commitment;
- a blocked topic or an individual grade (pattern `n/20`);
- reply text that itself reads as instructions;
- a language mismatch, or more than 200 words.

4.3 Sender authentication

A student's scope, and with it access to cohort-specific answers, is resolved from a roster by exact address. Because the `From` header is free text, the address is trusted only when the receiving server vouches for it (Figure 5). The system reads the top-most `Authentication-Results` header [1] and requires it to be written by the trusted server (`mx.google.com`). It then accepts either a DMARC pass whose `header.from` equals the From domain [2], or a DKIM pass whose signing domain is that domain or a parent of it [3]. Any `dmarc=fail` rejects the sender, and SPF alone is never sufficient [4].

The parser follows the RFC 8601 grammar for quoted strings and comments. Malformed headers, and comments glued to a value, fail closed. This closes the class of injection in which sender-controlled values echoed by the server forge a result [10]. An unauthenticated sender is not dropped. The email is shown to the owner with an explicit warning, but it receives an unknown scope (no memory reuse), no role weight, and an outlier mark that excludes automatic and bulk sending.

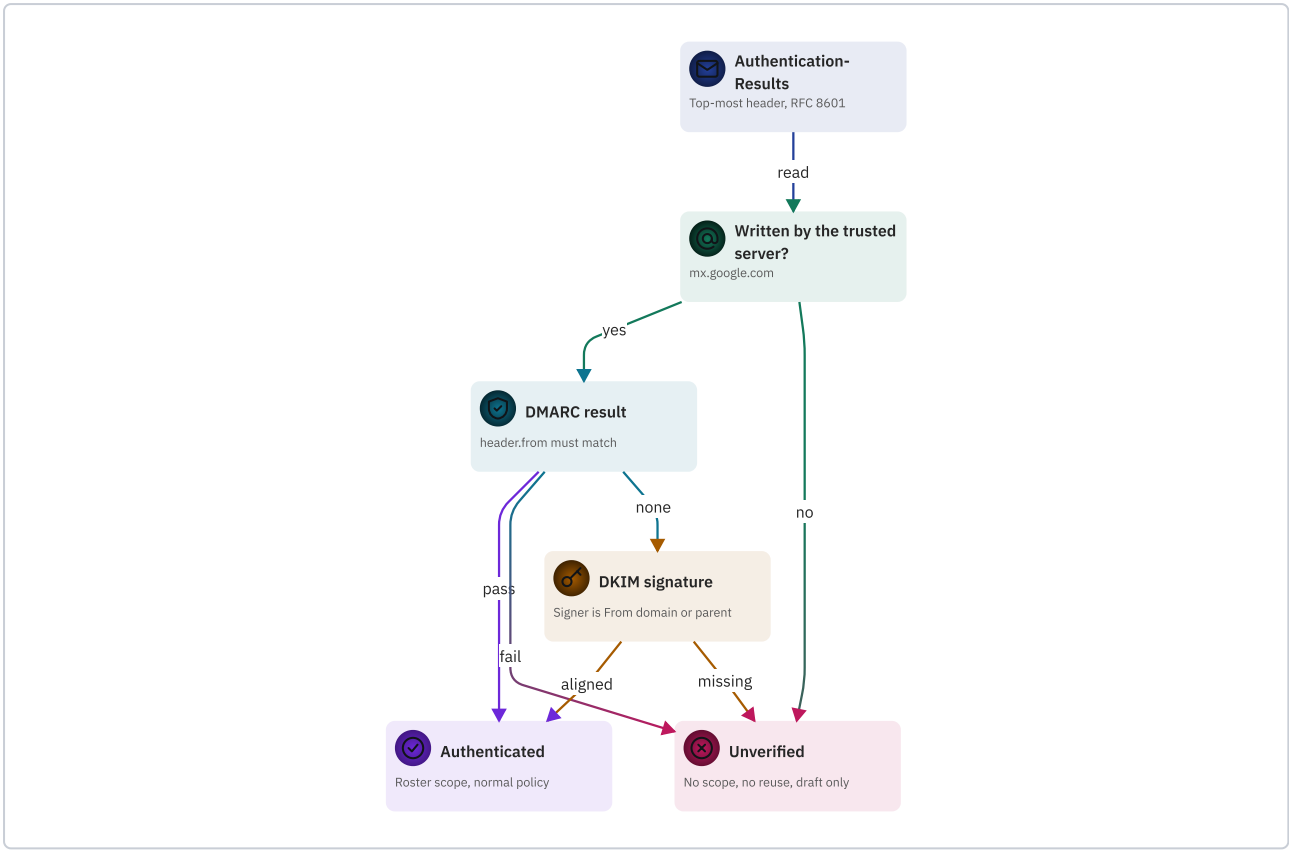


Figure 5. Sender-authentication decision. Source: `security/SenderAuthentication.java`; tests `SenderAuthenticationTest` (14 cases), `SenderAuthImportTest`.

4.4 Owner panel and secrets

The panel listens on `127.0.0.1` only. Login uses a PBKDF2-HMAC-SHA256 hash with 600,000 iterations [7]; without a configured hash, every login is refused. After five failures in fifteen minutes, all attempts are refused. Every form carries a CSRF token, and the content security policy forbids all scripts (`default-src 'none'; style-src 'self'`). The Gmail integration uses OAuth 2.0 with PKCE [9], reads only allow-listed senders and only creates drafts; the mailbox package contains no send call, which a test enforces. The refresh token and client secret are written through an owner-only temporary file and an atomic move. That means POSIX mode `600` / `700`, or a Windows ACL naming only the current user.

5 Privacy and compliance

Student emails are personal data under the GDPR, which requires data minimisation and data protection by design [8]. The implementation addresses this structurally (Table 3). Models must be local, and a non-local model address stops the application at start-up unless explicitly permitted. Filtered emails keep metadata but not their body. Facts marked personal are never reused.

Table 3. Privacy controls and their defaults.

Control	Default	Where enforced
Local model endpoint	`127.0.0.1:11434`; remote refused	`StartupConfigCheck`
Prompt and response logging	Disabled	`application.properties`
Email addresses in logs	Redacted to `[email]`	`PiiLogFilter`
Retention of finished emails	365 days, then erased	`Retention` (nightly)
Mail read in Gmail mode	Allow-listed senders only; empty list reads nothing	`GmailMailbox`
Fact freshness	Reused directly up to 90 days, then proposed for review	`ResolverAgent`
AI-assistance disclosure	Enabled in every reply	`WriterAgent`

6 Evaluation

The build runs 232 automated tests, with no failures at the reported commit (Figure 6b). Agent behaviour is tested through scripted models that pass through the real output parser. The attack corpus (Table 4) runs end to end without a model on every build, and its 17 attacks span six OWASP categories (Figure 6a). Two harmless control emails must still receive their replies, which guards against controls that are too strict. A separate evaluation set of 35 labelled emails measures triage and review against the local models.

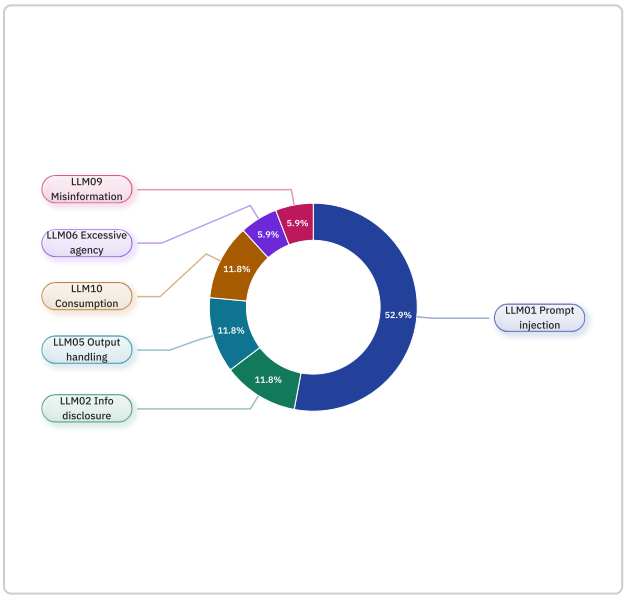


Figure 6a. Attack corpus by primary OWASP LLM category (n = 17). Source: `attacks/corpus.json`.

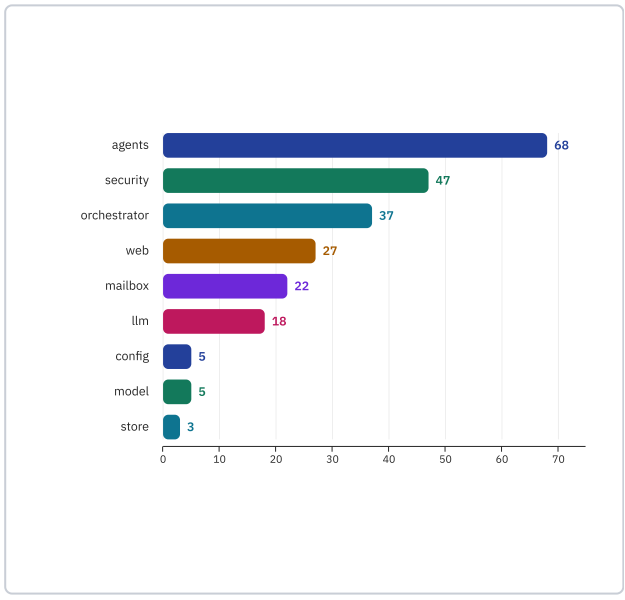


Figure 6b. Automated tests by package (n = 232). Source: Surefire reports of `./mvnw verify`.

Table 4. End-to-end attack corpus and the outcome each test requires. All pass.

ID	Attack	OWASP	Required outcome
A01	Direct instruction override	LLM01	Payload blocked
A02	Polite steering without trigger words	LLM01	Payload blocked
A03	Exfiltration link	LLM01/02	Payload blocked
A04	Contact swap	LLM01	Payload blocked
A05	Hidden HTML instruction	LLM01	Payload blocked
A06	Invisible Unicode	LLM01	Payload blocked
A07	False commitment	LLM09	Payload blocked
A08	Data leak request	LLM02	Payload blocked
A09	Impersonation	LLM01	Payload blocked
A10	Instruction relay to the next reader	LLM01	Payload blocked
A11	Markup in the reply	LLM05	Payload blocked
A12	Flooding the reply	LLM10	Payload blocked
A13	Header injection in subject	LLM05	Subject kept on one line
A14	Sensitive topic hidden as a question	LLM02	Not sent
A15	Scope spoofing	LLM06	No other cohort's answer
A16	Auto-reply loop	LLM10	Filtered
A17	French paraphrase of a cancellation	LLM01	Payload blocked
B1, B2	Benign control emails	–	Sent

The sender-authentication work was developed test-first and then examined by an independent reviewer. The reviewer found no critical issues and four important ones: a parser blind to quoted strings, a Windows principal lookup that could fail, missing visibility in the panel, and untested importer wiring. One minor issue was regraded as important: the mailbox type was chosen at build time but checked at run time. Each was fixed with a test that failed first. The importer tests were additionally validated by mutation: removing the line under test made both fail.

7 Limitations and future work

- **Forged mail and rate limits.** The per-sender hourly limit is keyed on the `From` address, so forged mail counts against the real student's limit.
- **Same-domain impersonation.** The `From` parser can take an address from a comment. A sender with valid DMARC for the same domain could therefore pose as a classmate. A strict address grammar is the planned fix.
- **Unsigned domains.** Mail from a domain without aligned DKIM or a DMARC record is always unverified. It is still handled, but it loses grouping and fact reuse until the domain enables signing.
- **Forwarded mail.** ARC chains are not evaluated, so legitimately forwarded mail is treated as unverified. This only produces false negatives.
- **Clarification notices.** They do not yet consider the unverified flag. This is reachable only outside Gmail mode, with every automatic-sending setting enabled.

8 Conclusion

The Email Response Agent shows that the useful parts of language models can be kept while their decision power stays narrow. Models classify, cluster and write. Code decides routing, trust and sending, and the database enforces the invariants that matter most. The input-slot abstraction reduces the owner's workload to answering each distinct question once. The layered, deterministic-first security design limits what a compromised or misled model can achieve. The remaining risks are known, bounded and documented.

References

1. M. Kucherawy. *Message Header Field for Indicating Message Authentication Status.* RFC 8601, IETF, 2019.
2. M. Kucherawy, E. Zwicky. *Domain-based Message Authentication, Reporting, and Conformance (DMARC).* RFC 7489, IETF, 2015.
3. D. Crocker, T. Hansen, M. Kucherawy. *DomainKeys Identified Mail (DKIM) Signatures.* RFC 6376, IETF, 2011.
4. S. Kitterman. *Sender Policy Framework (SPF) for Authorizing Use of Domains in Email.* RFC 7208, IETF, 2014.
5. K. Moore. *Recommendations for Automatic Responses to Electronic Mail.* RFC 3834, IETF, 2004.
6. OWASP Foundation. *OWASP Top 10 for Large Language Model Applications.* Version 2025.
7. OWASP Foundation. *Password Storage Cheat Sheet.* OWASP Cheat Sheet Series, 2023.
8. European Parliament and Council. *Regulation (EU) 2016/679 (General Data Protection Regulation)*, Articles 5 and 25. 2016.
9. N. Sakimura, J. Bradley, N. Agarwal. *Proof Key for Code Exchange by OAuth Public Clients.* RFC 7636, IETF, 2015.
10. J. Chen, V. Paxson, J. Jiang. *Composition Kills: A Case Study of Email Sender Authentication.* USENIX Security Symposium, 2020.
11. *Email Response Agent — System Design.* Project design document, 2026.